

SESSION 18

Maps & Geolocation API

Topics Covered: Google Maps API, Geocoding (address -> latitude, longitude), Embedding map in Django

Activities: Show doctor clinic location on map, Search doctors by distance

Task 1: Register for Google Maps API key and write Python script that takes address ('IIM Ahmedabad, Gujarat') and prints latitude and longitude using Google Geocoding API.

Explanation & Approach: *Queries Google Maps Geocoding REST API endpoint.*

```
import requests

def geocode_address(address, api_key):
    url =
    f"https://maps.googleapis.com/maps/api/geocode/json?address={address}&key={api_key}"
    response = requests.get(url).json()

    if response['status'] == 'OK':
        location = response['results'][0]['geometry']['location']
        print(f"Address: {address}")
        print(f"Latitude: {location['lat']}")
        print(f"Longitude: {location['lng']}")
        return location['lat'], location['lng']
    else:
        print("Geocoding failed:", response['status'])
        return None, None

# Example Usage:
# geocode_address("IIM Ahmedabad, Gujarat", "YOUR_API_KEY")
```

Task 2: Create Django view show_restaurant_location that takes restaurant address and displays web page with embedded Google Map.

Explanation & Approach: *Renders Google Maps Embed iframe passing dynamic latitude and longitude values.*

```
# views.py
from django.shortcuts import render

def show_restaurant_location(request):
    context = {
        'restaurant_name': 'Toscano - UB City',
        'lat': 12.9716,
        'lng': 77.5946,
        'api_key': 'YOUR_GOOGLE_MAPS_API_KEY'
    }
    return render(request, 'maps/restaurant_map.html', context)

<!-- templates/maps/restaurant_map.html -->
```

```

<h3>{{ restaurant_name }} Location</h3>
<iframe
  width="600"
  height="450"
  style="border:0"
  loading="lazy"
  allowfullscreen
  src="https://www.google.com/maps/embed/v1/place?key={{ api_key }}&q={{ lat }},{{ lng }}">
</iframe>

```

Task 3: Write Python function `find_nearby_cafes(user_lat, user_lng, cafes)` returning cafes within 3 km using Haversine formula without geolocation libraries.

Explanation & Approach: *Calculates great-circle distance between two lat/lng points using spherical trigonometry.*

```

import math

def haversine(lat1, lon1, lat2, lon2):
    R = 6371.0 # Earth radius in kilometers
    dlat = math.radians(lat2 - lat1)
    dlon = math.radians(lon2 - lon1)
    a = math.sin(dlat / 2)**2 + math.cos(math.radians(lat1)) * math.cos(math.radians(lat2))
    * math.sin(dlon / 2)**2
    c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
    return R * c # Distance in km

def find_nearby_cafes(user_lat, user_lng, cafes):
    nearby = []
    for cafe in cafes:
        dist = haversine(user_lat, user_lng, cafe['lat'], cafe['lng'])
        if dist <= 3.0: # Within 3 km
            cafe['distance_km'] = round(dist, 2)
            nearby.append(cafe)
    return sorted(nearby, key=lambda x: x['distance_km'])

# Test Sample
cafes_list = [
    {'name': 'Cafe Coffee Day', 'lat': 12.972, 'lng': 77.595},
    {'name': 'Starbucks', 'lat': 12.980, 'lng': 77.600},
    {'name': 'Far Away Cafe', 'lat': 13.100, 'lng': 77.800},
]
print("Nearby Cafes:", find_nearby_cafes(12.9716, 77.5946, cafes_list))

```

Task 4: Build Django view `search_by_distance` allowing users to see Flipkart pickup points (at least 5 hardcoded addresses) sorted by distance.

Explanation & Approach: *Renders sorted pickup points based on distance calculation.*

```

# views.py
from django.shortcuts import render

```

```
from .haversine import find_nearby_cafes

PICKUP_POINTS = [
    {'name': 'Flipkart Hub Indiranagar', 'lat': 12.9784, 'lng': 77.6408},
    {'name': 'Flipkart Hub Koramangala', 'lat': 12.9352, 'lng': 77.6245},
    {'name': 'Flipkart Hub MG Road', 'lat': 12.9756, 'lng': 77.6066},
    {'name': 'Flipkart Hub Whitefield', 'lat': 12.9698, 'lng': 77.7500},
    {'name': 'Flipkart Hub Jayanagar', 'lat': 12.9250, 'lng': 77.5938},
]

def search_by_distance(request):
    user_lat = float(request.GET.get('lat', 12.9716))
    user_lng = float(request.GET.get('lng', 77.5946))

    sorted_points = find_nearby_cafes(user_lat, user_lng, PICKUP_POINTS)
    return render(request, 'maps/pickup_points.html', {'points': sorted_points})
```